

The Active Scanner

Knocking on every door, politely

What this chapter is about

After Chapters 1 and 2, we know the subnets and the wiring. But we still don't have a complete *host* picture. A printer that never sent traffic through the firewall, a CCTV camera that the access switch sees only as "some MAC behind port 14", a developer's laptop powered on five minutes ago — these all need active investigation.

So we walk every subnet, send a polite probe to every IP, and record who responds. For each responder we figure out: *is it alive?*, *what ports are open?*, *what does it identify itself as?*, *what operating system?*. The output is a host inventory with operating system, hostname, and open services.

Real-life analogy. *You have the hallway map. You know there are 256 doors on floor 5. Now you walk along, knock on each door, listen for someone moving inside, and if they answer you ask "Hi, what's your name? What do you do here? What's your room number written above the door?" Most doors are empty. Some open immediately. A few open a tiny crack with a chain still on. You write all of it down. That's the active scanner.*

Words you'll see (and what they mean)

Active scanning. Sending probe packets to find out things. Contrast with "passive" (silently watching traffic). Active is faster and more complete; it's also visible to intrusion-detection systems, which is why we go gently.

ICMP (Internet Control Message Protocol). The protocol that `ping` uses. `ping 10.0.5.42` sends an ICMP echo-request and waits for an echo-reply. Many modern hosts (especially Windows) drop ICMP by default, so ICMP alone undercounts live hosts by 20–40%.

ARP probe. On the local broadcast domain, "ARP for 10.0.5.42" is more authoritative than ICMP — even hosts that block ICMP must answer ARP, otherwise they can't communicate at all. We use ARP probes whenever the target is on the same L2 segment as the collector.

"TCP ping". Connecting to a TCP port (typically 80, 443, 445) and seeing if it answers. A rejection ("port closed") still proves the host is alive — it's just not running that service. Catches hosts that block ICMP.

TCP connect scan. The simplest, no-privilege port scan: ask the OS to open a real TCP connection to a target port, and observe the result. Three outcomes: *open* (handshake completed), *closed* (RST received), *filtered* (no answer at all = firewall).

SYN scan / half-open scan. A faster scan that needs raw-socket privileges: send a SYN, wait for SYN-ACK, never complete the handshake. Tens of thousands of probes per second possible.

Banner grab. After connecting to an open port, read the first response without sending anything (or send a tiny request). Most services politely identify themselves: SSH says "SSH-2.0-OpenSSH_8.4", a web server says "Server: Apache/2.4.41", an SMB endpoint reveals the Windows build number, a TLS handshake reveals a certificate with the host's real name.

OS fingerprinting. Inferring the operating system from the way the host's TCP/IP stack behaves — the order of TCP options, the initial window size, retry timing. **Nmap's** OS database has 15+ years of curated signatures and gets ~85–95% accuracy. Pure-Go fingerprinting without Nmap signatures gets 30–50%. We will shell out to Nmap when accuracy matters.

SNMP probe. Try the SNMP "system" group on port 161 with default communities (`public` , `private`) and tenant-supplied creds. `sysObjectID.0` is a vendor/model identifier far more reliable than parsing `sysDescr` .

mDNS (Multicast DNS) / Bonjour. A protocol where Macs, printers, and IoT shout their names on a multicast group every few seconds. One multicast packet from us, and we learn `printer-3rd-floor.local` with no work.

NetBIOS Name Service. Old Windows protocol on UDP/137. Still useful: many corporate Windows networks have incomplete DNS, and a NetBIOS name query gives you the hostname

directly.

WS-Discovery (Web Services Discovery). A SOAP-over-multicast protocol used mostly by IP cameras, printers, and ONVIF devices. One probe packet → camera says "I am model X at this URL."

SSDP (Simple Service Discovery Protocol) / UPnP. Multicast probe used by smart-TVs, some media servers, IoT. Lower enterprise value than the above three, but cheap to add.

IDS/IPS (Intrusion Detection / Prevention System). Customer-side tooling that watches for "this looks like a port scanner." If we scan too fast or too aggressively, the customer's IDS will fire alerts and the customer's security team will call us at 3am. Going gently is non-negotiable.

conntrack. Linux kernel's connection-tracking table. If we open thousands of TCP connections per second, this fills up; the kernel starts dropping new connections. Bump `nf_conntrack_max` to 1,000,000 on the collector host.

Npcap. The library on Windows that lets us send raw packets. Required for SYN scan on Windows. *Note:* commercial OEM redistribution requires a paid licence — budget for it if you ship as SaaS on Windows collectors.

The probing pipeline (5 stages)

```
For each IP in the tenant's authorized CIDR:
```

```
┌ Stage 1 – Discovery (concurrent) ────────────
```

```
| ARP probe (if on local L2)
```

```
| ICMP echo
```

```
| TCP "ping" to 80, 443, 445 – catches ICMP-blockers
```

```
| → if any answers → host is alive; else → mark "no answer"
```

```
|
```

```
▼ | alive
```

```
┌ Stage 2 – Port probe ────────────
```

```
| TCP connect to top-20 ports
```

```
| if anything found → expand to top-100
```

```
| ports: 22 80 443 445 3389 161 554 631 902 8080 8443
```

```
|         + 135 23 9100 5060 1433 3306 5432 5985 5986
```

```
|
```

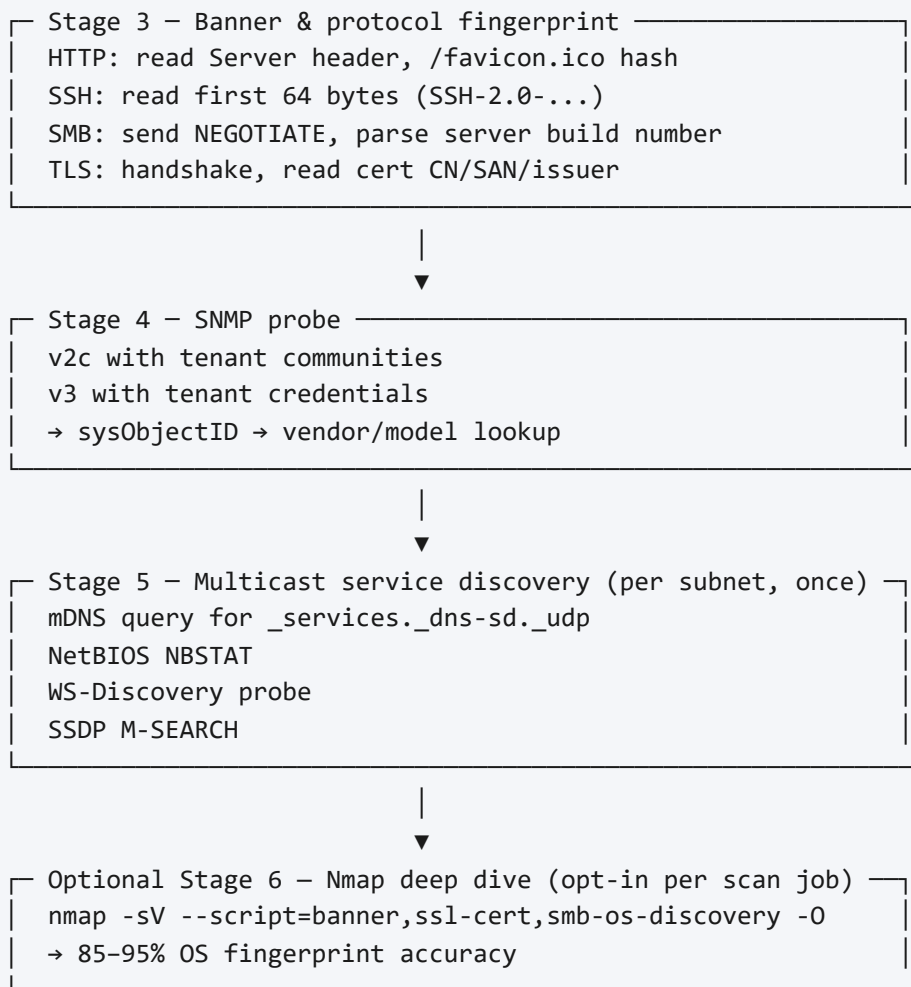
```
    | open ports
```

```
| Stage 1 - Discovery (concurrent) |  
| ARP probe (if on local L2)      |  
| ICMP echo                       |  
| TCP "ping" to 80, 443, 445 - catches ICMP-blockers |  
| → if any answers → host is alive; else → mark "no answer" |
```

| alive
▼

```
|----- Stage 2 - Port probe -----|  
| TCP connect to top-20 ports      |  
| if anything found → expand to top-100  
| ports: 22 80 443 445 3389 161 554 631 902 8080 8443  
|           + 135 23 9100 5060 1433 3306 5432 5985 5986
```

```
| open ports
```



What we expect to learn from each stage

Stage	What we learn	What it costs us
1 – Discovery	Live / dead	Tiny (3 packets/host)
2 – Ports	What services are running	11–100 packets/host, must be rate-capped
3 – Banners	Software versions, hostnames, certificate SAN	One full TCP connect per banner
4 – SNMP	Vendor, model, exact device type	3–4 UDP packets if community right; otherwise nothing
5 – Multicast	Printers, cameras, Macs, IoT names	One multicast packet per subnet

6 — Nmap	Accurate OS fingerprint, vuln hints	Heavy; opt-in per scan
----------	-------------------------------------	------------------------

Step-by-step plan

Step 1 — Privileges and platform setup

Active scanning is infrastructure work. Run the collector as:

- **Linux:** with `CAP_NET_RAW` capability granted (`setcap cap_net_raw=+ep ./itom-collector`) — lets us send raw ICMP and SYN packets without full root.
- **Windows:** as Administrator, with **Npcap** installed in WinPcap-API-compatible mode.

If neither is available, fall back to TCP-connect-only mode (no raw ICMP, no SYN scan). Slower and less complete, but works.

Step 2 — Stage 1 discovery

1. Use `github.com/prometheus-community/pro-bing` (the maintained fork of go-ping) for ICMP.
2. For ARP on local L2, use `github.com/google/gopacket` with raw `AF_PACKET` / Npcap.
3. For TCP ping, just `net.DialTimeout` with a 1.5-second timeout.
4. Run ARP, ICMP, and TCP-ping *concurrently*; mark host alive if any one succeeds.

Step 3 — Stage 2 port probe

1. Define a tiered port list. Top-20 first; only expand to top-100 if at least one port responded.
2. Worker pool: 500–1,000 goroutines per collector. Above that, the Go scheduler thrashes and conntrack fills up.
3. Per-host TCP connect timeout: 2 seconds.
4. Rate-cap: **1,000 packets/sec per /24**, with $\pm 25\%$ jitter. Higher rates trigger Palo Alto / Firepower IDS.

Step 4 — Stage 3 banner grab

1. For HTTP: send `GET / HTTP/1.0\r\nHost: 10.0.5.42\r\n\r\n`, read first 4 KB, parse `Server` header.
2. For SSH: just read 64 bytes after connect. The protocol mandates the banner is the first thing sent.

3. For TLS (443, 8443, 993, etc.): full TLS handshake with hostname=IP, accept any cert, parse the certificate's CN and SubjectAltName. *Internal hostnames hide here.*
4. For SMB on 445: send a NEGOTIATE_PROTOCOL_REQUEST, parse the response's `os` field. Free Windows OS fingerprint.

Step 5 — Stage 4 SNMP probe

1. For each tenant-configured community + the defaults `public` , `private` : try a single GET on `sysObjectID.0` with 1-second timeout.
2. If response, also fetch `sysName.0` , `sysDescr.0` .
3. Resolve `sysObjectID` against an embedded copy of the IANA Enterprise Numbers list to get vendor name (e.g., 1.3.6.1.4.1.9 = Cisco).

Step 6 — Stage 5 multicast (one per subnet)

1. **mDNS:** use `github.com/hashicorp/mdns` . Send one query for `_services._dns-sd._udp.local` , listen 3 seconds.
2. **NetBIOS:** ~50 lines of UDP/137. Send NBSTAT query "*", parse the workstation/server name list.
3. **WS-Discovery:** SOAP envelope to `239.255.255.250:3702` . Listen 3 seconds.
4. **SSDP:** "M-SEARCH * HTTP/1.1" to `239.255.255.250:1900` . Listen 2 seconds.

Step 7 — Optional Nmap orchestration

Behind a feature flag `scan_job.deep_fingerprint` :

1. Use `github.com/Ullaakut/nmap/v3` to run `nmap -sV -O --script=banner,ssl-cert,smb-os-discovery` against alive hosts.
2. Parse the XML output back into our observation model.
3. Cap at, say, 500 hosts per scan run; this is heavy.

Nmap binary is ~30 MB. Bundle it with the collector or detect-and-skip if missing.

Step 8 — Multi-tenant guards (every probe goes through this)

Before any single packet leaves the collector, the target IP is checked against the signed CIDR allowlist from Chapter 0. No allowlist match → no packet. Audit row written. This is the single most important safety rail in the whole system.

Performance: what's realistic?

On a 4 vCPU, 1 Gbps collector VM scanning a /16 (65,536 addresses):

Stage	Time	Notes
Stage 1 discovery	5–10 minutes	1,000–2,000 packets/sec; 5–10% alive on average
Stage 2 port probe (top 11) on alive	15–25 minutes	500–1,000 worker goroutines, 2-second timeout
Stage 3 banners	5–10 minutes	Full TCP connect per banner; rate-bound
Stage 4 SNMP probe	3–5 minutes	UDP, parallel, cheap
Stage 5 multicast (256 subnets in /16)	15 minutes (sequential)	One probe per subnet, 3–5 sec listen each
Stage 6 Nmap deep (opt-in, 500 hosts)	30–90 minutes	Don't run this every cycle

Total non-Nmap pass on a /16: about **45–60 minutes**. A full /24 is comfortably under 2 minutes. This sets a sensible default cadence: daily for /16+, hourly for /24-sized branches.

How we test Chapter 3

Lab setup. Build a known subnet with a mix of:

- A Linux server (Ubuntu) — should fingerprint via SSH banner + ICMP-replies.
- A Windows server — should fingerprint via SMB negotiate + NetBIOS.
- A Mac laptop — should appear via mDNS.
- A network printer — should appear via WS-Discovery + SNMP sysObjectID.
- A Cisco switch — should appear via SNMP.
- An empty IP — must be marked dead, must never produce a fake observation.

Test 1 — ICMP-blocked host still found.

1. On the Windows server, enable Windows Firewall (blocks ICMP by default).

2. Run discovery; confirm host is still marked alive (TCP ping to 445 succeeded).

Test 2 — Banner accuracy.

1. SSH banner of the Linux server appears verbatim in the `open_port` row.
2. SMB banner contains the Windows OS build (e.g., "Windows Server 2019 Build 17763").
3. TLS cert SAN of an internal HTTPS service appears as a discovered hostname.

Test 3 — Multicast discovery.

1. Power on a Mac in the lab subnet. Confirm its `.local` name appears via mDNS.
2. Power on the network printer. Confirm WS-Discovery picks up its model.

Test 4 — Rate cap.

1. Run a full /24 scan with rate cap set to 100 pps.
2. Use `tcpdump` to count packets per second; assert never exceeds 100 + jitter.

Test 5 — Allowlist enforcement.

1. Configure tenant allowlist as `10.0.0.0/8`.
2. Issue a scan job for `192.168.1.0/24`.
3. Confirm: zero packets sent to 192.168.x.x; `ScanJobError` returned; audit row written.

Test 6 — Optional Nmap tier.

1. With Nmap installed, enable `deep_fingerprint=true`.
2. Verify OS field on the Linux server contains "Linux 5.x" with high accuracy from Nmap output.

Things that can go wrong (and what to watch for)

IDS storm. Scanning at full speed on a customer's prod network can trigger their IDS into "we are under attack" mode. Always rate-cap, jitter, and warn the customer's network team

before the first scan.

Switch CPU spike from ARP storm. A fast subnet sweep generates an ARP burst. Older / cheaper switches can spike to 90% CPU. Cap rate per /24, monitor switch CPU during early lab runs.

Conntrack exhaustion. 1,000 worker goroutines × 2-sec timeouts × multiple subnets = quickly hits the default `nf_conntrack_max=65536`. Bump it to 1,000,000 and tune `nf_conntrack_tcp_timeout_*` values down.

Treating "no response" as "no host". Could equally be a firewall, a host with strict iptables, or just a busy host. Mark it "no-response", not "dead." Dead requires explicit RST or unreachable.

Banner-grab to the wrong protocol. Don't send HTTP request bytes to port 22 — you'll just confuse a sensitive SSH server and might trigger fail2ban. Map ports to protocol probers; never one-size-fits-all.

Default SNMP communities firing alerts. Some IDS rules flag "public from outside" as a scanner signature. Try tenant-supplied creds first; only fall back to defaults if explicitly enabled per tenant.

Privilege drop after first packet. On Linux, if you start as root then drop privs, raw sockets sometimes lose `CAP_NET_RAW`. Test the privilege-drop path explicitly.

Nmap binary as a security flag. Some endpoint security agents flag `nmap.exe` as a hacking tool and quarantine it. Document this for customers; offer a non-Nmap mode.

Goroutine leak on context cancel. If a scan is cancelled mid-sweep, every in-flight TCP-connect goroutine must respect the context. Audit this carefully — leaked goroutines pile up and eat memory over weeks.

Exit checklist (you can move to Chapter 4 only when ALL are true)

- ☐ Stages 1–5 implemented and tested against a mixed lab subnet (Linux/Windows/Mac/printer/switch).
- ☐ ICMP-blocked Windows host still discovered via TCP ping.
- ☐ Banner grab populates `open_port.banner` for HTTP, SSH, SMB, TLS.
- ☐ TLS cert SAN appears as a discovered hostname.
- ☐ mDNS, NetBIOS, WS-Discovery each produce at least one matched observation in lab.
- ☐ Rate cap enforced and verified with packet capture.
- ☐ Allowlist refusal verified — out-of-range targets produce zero packets.
- ☐ Conntrack / file-descriptor / goroutine counts steady-state during a 4-hour soak test.
- ☐ Nmap deep tier behind a feature flag, with graceful skip when binary missing.
- ☐ Documented privilege requirements (CAP_NET_RAW on Linux, Administrator + Npcap on Windows).

Recap card — Chapter 3 in 5 sentences

1. Walk every IP in the tenant's authorized CIDRs and probe in five stages: discovery → ports → banners → SNMP → multicast.
2. ICMP alone undercounts by 20–40%; always run ARP + ICMP + TCP-ping concurrently in stage 1.
3. Banner grabs (HTTP, SSH, SMB-negotiate, TLS cert) plus SNMP sysObjectID give 70–85% device identification with pure Go.
4. Multicast (mDNS, NetBIOS, WS-Discovery) costs one packet per subnet and catches 10–20% of devices the others miss — Macs, printers, cameras, IoT.
5. Always rate-cap at ~1,000 packets/sec per /24 with jitter; check the signed CIDR allowlist on every single packet; offer Nmap as opt-in for deep OS fingerprint.